講義 11：多表 JOIN 與 CORRESPONDING FIELDS

> 對應練習：ex11 | 答案程式：`ZR_TR11_JOIN`

本講重點

- 為什麼要 JOIN：報表幾乎都要跨表撈欄位
- `INNER JOIN` 與 `LEFT OUTER JOIN` 的差別
- 語法細節：別名 `AS`、欄位前綴 `~`、`ON` 條件
- `INTO CORRESPONDING FIELDS OF TABLE`：依欄位名對應
- `FOR ALL ENTRIES` 簡介與著名的空表陷阱
- 效能觀念：JOIN vs 迴圈中 SELECT

1. 為什麼要 JOIN

需求：「列出每個航班，**含航空公司名稱**」。航班在 SFLIGHT，公司名稱在 SCARR——單表 SELECT 拿不齊。最糟的寫法是迴圈裡逐筆查：

```
* 反面教材：N 筆航班 = N 次資料庫往返，大表直接跑不動
LOOP AT gt_flights INTO gs_flight.
  SELECT SINGLE carrname FROM scarr INTO gv_name
    WHERE carrid = gs_flight-carrid.
ENDLOOP.
```

JOIN 讓資料庫**一次**把兩張表串好回傳——這是本講存在的理由。

2. INNER JOIN

```
TYPES: BEGIN OF ty_flight,
         carrid   TYPE sflight-carrid,
         connid   TYPE sflight-connid,
         fldate   TYPE sflight-fldate,
         carrname TYPE scarr-carrname,     " 來自另一張表的欄位
       END OF ty_flight.
DATA gt_flights TYPE STANDARD TABLE OF ty_flight.

SELECT f~carrid f~connid f~fldate c~carrname
  INTO CORRESPONDING FIELDS OF TABLE gt_flights
  FROM sflight AS f
  INNER JOIN scarr AS c ON f~carrid = c~carrid
  WHERE f~fldate >= '20260101'.
```

語法要點：

- `AS f` / `AS c`：表別名。之後**每個欄位都要標明來源**：`f~carrid`、`c~carrname`（波浪號 `~`，不是 `-`）。

- ON 是**串表條件**（兩表怎麼對上），WHERE 是**過濾條件**——語意不同，別混在一起。ON 可以多條件：ON f~carrid = c~carrid AND f~connid = c~connid。
- INNER JOIN 的語意：**兩邊都對得上才輸出**。SFLIGHT 有但 SCARR 沒有的公司代碼，那些航班整筆消失。

新式寫法對照（7.40 之後）：JOIN 加行內宣告（Inline Declaration）

```
* 新式寫法：不用先寫 TYPES / DATA
SELECT f~carrid, f~connid, f~fldate, c~carrname
  FROM sflight AS f
  INNER JOIN scarr AS c ON f~carrid = c~carrid
  WHERE f~fldate >= '20260101'
  INTO TABLE @DATA(gt_flights).
```

- 規則跟講義 6 §3.6 一樣：欄位用逗號、INTO 放最後、宿主變數（Host Variable）加 @（例如 WHERE f~carrid IN @s_carrid）。
- 最大的好處在 JOIN：傳統寫法要先定義一個「東拼西湊」的結構 ty_flight，新式寫法用 @DATA(gt_flights) 讓系統依 SELECT 清單自動產生結構，欄位名就是 carrid、connid、fldate、carrname（不含別名前綴），後面照樣寫 gs_flight-carrname。
- 結果要放進既有的內表（Internal Table）時，照樣可以寫 INTO CORRESPONDING FIELDS OF TABLE @gt_flights。

練習 11 的新式寫法版本見答案程式 ZR_TR11_JOIN_NEW（INNER JOIN、LEFT OUTER JOIN 都改用 @DATA( )，輸出跟原版逐行比對相同）。

2.1 另一種寫法：Database View（把 JOIN 定義在 DDIC）

同一組 JOIN 如果很多支程式都要用，可以在 SE11 建一個 **Database View**，把「哪幾張表、怎麼連、要哪些欄位」定義一次，程式就像讀一張表一樣讀它。系統標準的 SFLIGHTS 就是這種 View（2026-09-29 查系統定義表 DD26S / DD28S）：

項目	SFLIGHTS 的內容
基礎表	SCARR、SPFLI、SFLIGHT
JOIN 條件	SCARR-CARRID = SPFLI-CARRID、SPFLI-CARRID = SFLIGHT-CARRID、SPFLI-CONNID = SFLIGHT-CONNID（再加各表 MANDT 相等）
欄位	CARRID、CARRNAME、CONNID、COUNTRYFR、CITYFROM、AIRPFROM、COUNTRYTO、CITYTO、AIRPTO、FLDATE、SEATSMAX、SEATSOCC

程式讀它，不用寫 JOIN：

```
DATA gt_flights TYPE STANDARD TABLE OF sflights.     " View 本身就是一個結構型別

SELECT carrid carrname connid cityfrom cityto fldate seatsmax seatsocc
  FROM sflights
```

```
  INTO CORRESPONDING FIELDS OF TABLE gt_flights
  WHERE carrid = 'LH'.
```

	Open SQL JOIN（本節上面）	**Database View**
JOIN 定義在哪	每支程式自己寫	DDIC，寫一次大家共用
JOIN 種類	INNER、LEFT OUTER 都可以	**只有 INNER JOIN**
可以做什麼	讀取	讀取；SE16 也能直接看資料；只有一張基礎表時還能寫入
建立方式	寫在程式裡	SE11 → View → Database view：Table/Join Conditions 填表與條件（有外鍵（Foreign Key）時可按 Relationships 帶出）→ View Fields 選欄位 → 啟用

選擇的原則：**只有一支程式用，直接在程式裡 JOIN；很多程式都要同一組 JOIN，才建 View**。S/4HANA 之後新開發改用 **CDS View**（CDS 課程），它能做 OUTER JOIN、計算欄位、關聯等 Database View 做不到的事；傳統 Database View 主要在維護舊程式時會遇到。

3. LEFT OUTER JOIN

「左表全留，右表對不上就給初始值」：

```
SELECT c~carrid c~carrname f~connid f~fldate
  INTO CORRESPONDING FIELDS OF TABLE gt_result
  FROM scarr AS c
  LEFT OUTER JOIN sflight AS f ON c~carrid = f~carrid.
```

結果：**每家**航空公司都在（左表 SCARR 全留）；沒有任何航班的公司，`connid`/`fldate` 是初始值（空白/00000000）。

選擇原則：「沒對到的要不要出現在報表上？」要 → LEFT OUTER；不要 → INNER。例如「各公司營收統計，沒航班的公司也要列出 0」就是 LEFT OUTER 的場景。

> 回顧講義 25：當時還沒學 JOIN，旺季加成報表是「三張表各自讀進內表，再用 `READ TABLE` 對照，找不到加成設定就維持初始值」。那段 `LOOP` 加兩個 `READ TABLE`，用一句 `LEFT OUTER JOIN ztr25_surchg` 就能取代——沒設定的航空公司，加成欄位是初始值，照樣出現在結果裡。

限制：LEFT OUTER JOIN 的 WHERE 對**右表**欄位下條件會把 NULL 列過濾掉，效果變回 INNER——右表條件盡量放在 ON。

新式寫法對照（7.40 之後）：`COALESCE`（Coalesce Function），沒對到的欄位給預設值

LEFT OUTER JOIN 沒對到時，右表欄位在資料庫裡是 NULL，撈回 ABAP 變成初始值（空白）。新式寫法可以用 `COALESCE` 直接換成看得懂的預設值：

```
SELECT a~carrid, coalesce( b~connid, '----' ) AS connid
  FROM scarr AS a
  LEFT OUTER JOIN spfli AS b ON a~carrid = b~carrid
  INTO TABLE @DATA(gt_carr).
```

`coalesce( 欄位, 預設值 )`：欄位有值就用欄位，是 NULL 就用預設值。本系統 `SCARR` 有一家沒有航線的 `AB`，結果是 `AB ----`；有航線的 `LH` 照常列出 `0400`、`0401`……（2026-10-08 用驗證程式 `ZR_TR06_OSQL_CHK` 在本系統 7.54 實測）。

4. INTO 的兩種對應方式

寫法	對應規則	風險
`INTO TABLE gt`	依**順序**：第 1 個欄位塞結構第 1 欄…	順序錯 = 資料錯位（不報錯）
`INTO CORRESPONDING FIELDS OF TABLE gt`	依**欄位名**：同名才塞	欄名拼錯 = 該欄默默留空

JOIN 的結果結構是自訂的、欄位東拼西湊，**建議一律用 CORRESPONDING FIELDS**：順序自由、可讀性高；代價是欄位名必須跟 SELECT 清單一致（別名欄位可用 `AS`：`SELECT f~price AS ticket_price ...`）。

`INTO` 與 `APPENDING`：`INTO TABLE` 會先清空內表再放結果；`APPENDING TABLE`（或 `APPENDING CORRESPONDING FIELDS OF TABLE`）則是**接在內表現有資料後面**，原本的資料保留。講義 13 的實戰案例 ZRFI0004 就用它把幾張結構相近的表讀進同一種內表：

```
SELECT bukrs belnr gjahr buzei ... saknr AS hkont ...   " 這張表的科目欄叫 SAKNR，用 AS 改名對上內表的 HKONT
  APPENDING CORRESPONDING FIELDS OF TABLE t_docps
  FROM vbsegs
  FOR ALL ENTRIES IN t_dochd
  WHERE ausbk EQ t_dochd-bukrs AND belnr EQ t_dochd-belnr AND gjahr EQ t_dochd-gjahr.
```

`AS` 在這裡的用途跟 JOIN 一樣：來源欄位名跟內表欄位名不同時，用別名讓 `CORRESPONDING FIELDS` 對得上。

新式寫法對照（7.50 之後）：`UNION`（集合運算，Set Operator），一句合併兩個 SELECT

`APPENDING` 是「執行兩次 SELECT，結果接在同一張內表」；`UNION` 是在資料庫端把兩個 SELECT 的結果合成一份再回傳：

```
SELECT carrid, connid FROM spfli WHERE cityfrom = 'FRANKFURT'
UNION
SELECT carrid, connid FROM spfli WHERE cityto = 'FRANKFURT'
  INTO TABLE @DATA(gt_frankfurt).
```

- 兩個 SELECT 的欄位數量、順序、型別要對得上；`INTO` 寫在最後一個 SELECT 之後。
- `UNION` 會去掉完全重複的列；要保留重複的列寫 `UNION ALL`。

- 本系統實測：從法蘭克福出發或抵達的航線共 15 筆（2026-10-08 用驗證程式 `ZR_TR06_OSQL_CHK` 在本系統 7.54 實測）。

5. FOR ALL ENTRIES（先認識，維護會遇到）

JOIN 之外的另一種跨表手法：先撈第一張表進內表，再用內表內容當第二張表的條件：

```
IF gt_flights IS NOT INITIAL.          " 這個 IF 是保命符！
  SELECT carrid carrname FROM scarr
    INTO TABLE gt_carriers
    FOR ALL ENTRIES IN gt_flights
    WHERE carrid = gt_flights-carrid.
ENDIF.
```

兩個必知：

- **空表陷阱**：`gt_flights` 是空的時，FOR ALL ENTRIES 的 WHERE 整個失效 = **全表撈回**。前面那個 `IF ... IS NOT INITIAL` 不是可選的。（ZRFI0004 沒有在 FOR ALL ENTRIES 前面寫 IF，是因為更早讀表頭時，查無資料就已經 `STOP` 了，驅動內表保證不會是空的。防呆一定要有，只是可以放在更前面。）
- 結果會自動去除完全重複的列。

新程式優先 JOIN；FOR ALL ENTRIES 用在 JOIN 不方便的場景（如來源是加工過的內表），舊程式裡極常見。

新式寫法對照（7.40 之後）：FOR ALL ENTRIES

```
IF gt_keys IS NOT INITIAL.                          " 空表陷阱一樣存在
  SELECT carrid, connid, fldate
    FROM sflight
    FOR ALL ENTRIES IN @gt_keys
    WHERE carrid = @gt_keys-carrid
      AND connid = @gt_keys-connid
    INTO TABLE @DATA(gt_flights).
ENDIF.
```

規則跟其他新式寫法一樣：驅動內表和它的欄位都是宿主變數，要加 `@`（`IN @gt_keys`、`@gt_keys-carrid`）。空表會全表撈回的陷阱不變，前面的 `IF` 照樣不能少。

6. 效能觀念小結

寫法	資料庫往返	評價
LOOP 內 SELECT SINGLE	N 次	禁止（測試資料看不出慢，上線就爆）
SELECT + FOR ALL ENTRIES	2 次	可用，記得空表防呆
INNER / LEFT OUTER JOIN	1 次	首選

7. 常見錯誤與陷阱

症狀	原因
欄位前綴報語法錯誤	JOIN 裡欄位要用 `f~carrid`（`~`），不是 `-`
某些資料整筆不見	INNER JOIN 對不上就丟——確認是否該用 LEFT OUTER
CORRESPONDING 後某欄全空	結構欄位名跟 SELECT 欄位名（或 AS 別名）不一致
LEFT OUTER 加了右表 WHERE 後行為像 INNER	右表條件要放 ON，不放 WHERE
FOR ALL ENTRIES 慢到懷疑人生	驅動內表是空的，全表撈回——補 IS NOT INITIAL 檢查

8. 課堂練習

完成 ex11：INNER JOIN 撈「航班 + 公司名稱」、LEFT OUTER JOIN 觀察沒航班的公司、比較兩種 INTO 的行為差異。